



Canon Project

≡ 작업 영역	Backend Flask Python 딥러닝 컴퓨터비전
📅 작업기간	@2024년 3월 11일 → 2024년 5월 30일



[프로젝트 소개]

정확한 기종 식별과 부품 부착 여부는 대오더라벨 품질 관리의 핵심입니다. 본 프로젝트에서는 라벨의 구분번호를 기준으로 기종을 분류하고, 템플릿 매칭 기술을 활용해 정규 부품의 부착 여부를 자동으로 검사하는 시스템을 개발했습니다. 이미지 처리 기술과 바코드 디코딩을 통합하여 체계적이고 효율적인 검사 프로세스를 구현하였습니다.

[역할]

- 4인 팀의 프로젝트에서 **백엔드 개발** 담당
- **검사 프로세스 설계 및 시스템 구축**

- 웹 통신 기반 시뮬레이션 기능 구현
- 대오더라벨 이미지 자동 추출 로직 개발
- 검사 결과 시각화를 위한 GUI 제작

[성과]

- 대오더라벨을 인식하고 **기종별 핸들러 자동 호출**
- **SIFT, Template Matching, One-Class SVM, CNN** 기반의 부품 검사 알고리즘 구현
- 다양한 기종에 유연하게 대응 가능한 검사 프로세스 구축
- GUI를 통해 검사 결과를 직관적으로 시연 가능하게 구현

대오더라벨 바코드 디코딩



대오더 라벨 인식 프로젝트에서는 제품 라벨 이미지로부터 텍스트 데이터를 정확하게 추출하는 과정을 구현했습니다. 먼저 Canny 알고리즘과 허프 변환을 통해 라벨의 기울어진 각도를 계산한 후, 이를 기준으로 이미지를 정렬하였습니다. 이후 회전된 이미지에서 바코드 영역을 인식하고 decode 함수를 활용해 텍스트 데이터를 추출하여 최종적으로 라벨 정보를 획득할 수 있도록 했습니다.

Template Matching

[수정 전]



- 템플릿 매칭 시 회전 각도를 변화시키며 입력되는 이미지 데이터에 대해 템플릿 매칭을 수행하였습니다. (전체 이미지 데이터 수: 1,014개)

회전 각도 범위	사각형이 그려진 이미지 수
-15° ~ +15°	721개
-45° ~ +45°	748개
-15° ~ +75°	760개

- 그러나 약 **25%의 이미지에서 체크포인트를 올바르게 인식하지 못하는** 문제가 발생하였습니다.
- **임계값을 상향 조정**하여 템플릿 매칭의 정확도를 높이하고자 하였으나, 이로 인해 **정상 제품조차 인식되지 않는** 경우가 빈번히 발생했습니다.
- **밝기 조절**을 통한 보정 기법도 시도했으나, **처리 시간이 증가**했고, 체크포인트 인식을 또 한 큰 개선을 보이지 않았습니다.
- 이와 같은 한계로 인해, **템플릿 매칭 단독으로는 안정적인 검사 시스템을 구축하기 어려움**을 인지하고, **추가적인 접근 방법의 도입 필요성**을 고민하게 되었습니다.

[수정 후]

프로젝트 진행 과정

Template Matching

step 1

[예시] 구분번호 C550(토너 지역 라벨: 일본/아시아) Step_1



05

Step 1. 이미지 전처리 및 정렬

1. 입력 이미지를 그레이스케일 변환 → 반전 → 팽창 과정을 통해 전처리합니다.
2. Otsu 이진화를 적용해 객체 윤곽을 추출하고,
3. 가장 큰 사각형 컨투어를 기준으로 라벨 위치를 탐색합니다.
4. 라벨이 포함된 영역을 수평으로 정렬하여 이후 매칭 정확도를 높입니다.

프로젝트 진행 과정

Template Matching

step 1

[예시] 3029C005AA

템플릿 이미지	
MATCHING_THRESHOLD	0.7
matched_images	1,010
failed_images	4
검출률	99.6%

[매칭 결과]



템플릿 이미지	
MATCHING_THRESHOLD	0.7
matched_images	2,165
failed_images	35
검출률	98.4%

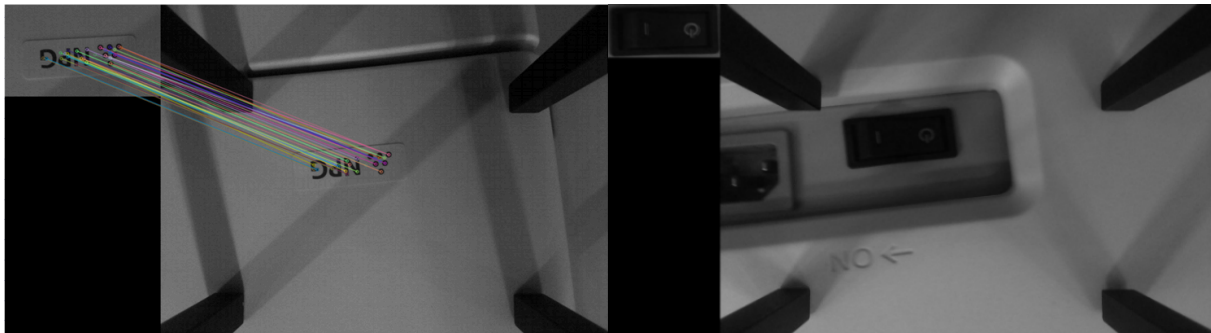
06

Step 2. 템플릿 매칭 수행

- 정렬된 이미지와 템플릿 이미지를 비교하여 유사도 기반 매칭을 수행하고,
- 일정 threshold 이상일 경우 해당 위치를 정확히 식별합니다.
- 예시 결과에서 각각 검출률 99.6%, 98.4%를 달성하여 높은 신뢰도를 확인할 수 있었습니다.

SIFT와 FLANN을 이용한 특징점 매칭

[수정 전]



SIFT와 FLANN을 활용하여 템플릿 이미지와 검사 이미지 간의 특징점 매칭을 수행하였습니다.

대부분의 경우에서 정상적으로 매칭이 이루어졌으나, 오른쪽 그림처럼 실제로 매칭이 되어야 하는 이미지임에도 불구하고 매칭이 실패하여 연결선이 나타나지 않는 사례가 발생하였습니다.

이러한 결과를 바탕으로 문제 상황을 분석하고, 다음과 같은 개선 사항을 도출하였습니다.

1. 매칭 정확도 향상 필요
2. 정상 이미지에서 매칭이 누락되는 경우 개선
3. 잘못된 위치 간 매칭 방지
4. 임계값 기준 재검토 필요

[수정 후]

이 문제를 해결하기 위해 기존 SIFT 기반 구조에 **템플릿 매칭 기법**을 결합하여 다음과 같은 흐름으로 개선하였습니다.



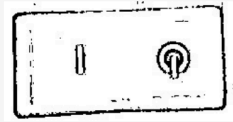
1. 전처리

- 검사 이미지와 템플릿 이미지를 **grayscale로 변환**하고,
- `cv2.adaptiveThreshold` 를 적용해 템플릿 매칭 성능을 향상시킬 수 있도록 **이진화 처리**하였습니다.

프로젝트 진행 과정

Template Matching & SIFT step 9

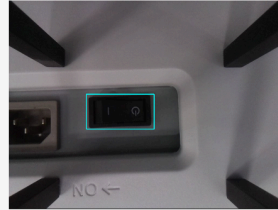
템플릿 이미지



템플릿 매칭 수행



[매칭 결과]



전처리된 이미지에서 템플릿 이미지와 일치하는 부분을 찾아 위치 식별

08

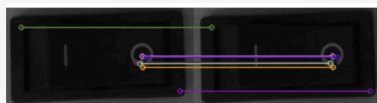
2. 템플릿 매칭 수행

- 이진화된 이미지에서 고정된 템플릿 이미지와 유사한 영역을 탐색 하였습니다.
- 매칭된 영역의 위치를 기준으로 관심 영역을 정확히 식별 하였습니다.

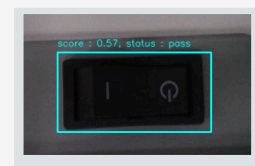
프로젝트 진행 과정

Template Matching & SIFT step 9

템플릿 이미지



[매칭 결과]



SIFT를 이용한 정밀 매칭 수행

매칭된 영역에서 SIFT 알고리즘을 사용하여 특징점을 검출

09

3. SIFT를 이용한 정밀 검출

- 템플릿 매칭으로 좁혀진 영역 내에서 SIFT 알고리즘을 적용하여 **더 정밀한 특징점 매칭** 수행 하였습니다.
- FLANN 매처와 Lowe's Ratio Test를 통해 **신뢰 높은 매치만 필터링** 하였습니다.

프로젝트 진행 과정

Template Matching & SIFT

step 9

[예시] 3029C009AA 데이터

결과

	step 1	step 2	step 3	step 4	step 5	step 6	step 7	step 8	step 9	step 11
pass	0	0	19	0	0	0	1	0	262	0
fail	272	272	253	272	272	272	271	272	10	272
검출률	0.00	0.00	0.07	0.00	0.00	0.00	0.00	0.00	0.96	0.00

10

개선 결과

- 템플릿 매칭 도입 이후, SIFT의 적용 영역을 국소화하여 **오탐 및 미탐 문제를 크게 줄였** 습니다.
- Step 9의 검출률은 **96%까지 향상**, 신뢰도 높은 매칭이 안정적으로 수행 되었습니다.
- 기존 "0개 매치 시 NG" 판정 기준도 검출률 데이터를 기반으로 보완 가능성 확보 하였 습니다.

OneClassSVM

프로젝트 진행 과정

OneClassSVM

step 6

[예시] 3029C009AA

EX) 3029C009AA_2EX36398_step_6



3029C009AA의 step 6 이미지를 VGG16 네트워크에 적합하도록 전처리

VGG16 모델 로드 : 분류 계층을 제외하고 특징 추출 부분만 사용

OneClass SVM을 사용하여 특징 벡터에 대해 학습을 수행 [$nu = 0.01$]

오류(이상치)의 비율 : 1%

새로운 이미지에 대해 같은 전처리 및 특징 추출 과정을 거쳐 OneClass SVM 모델로 예측을 수행



예측 결과에 따라 이미지가 'Pass' 또는 'Fail'로 분류

11

OneClassSVM을 활용한 이상 이미지 판별

Step 6에서는 제품 내부 구조가 복잡하고, 불량 사례가 희귀하거나 다양해 명확한 기준이 존재하지 않는 문제가 있었습니다.

일반적인 분류 모델을 학습시키기엔 불량 데이터 수집이 어렵고, 패턴도 매우 제한적이기 때문에, 정상 데이터만으로도 이상 여부를 판단할 수 있는 OneClassSVM 기반 접근을 선택하였습니다.

적용 방식

- 사전 학습된 VGG16 모델의 특징 추출 계층을 활용하여, step 6 이미지에서 고차원 특징 벡터를 추출하였습니다.
- 추출된 벡터들을 기반으로 One-Class SVM 모델을 학습하여, 정상 이미지 분포를 정의하였습니다. ($nu = 0.01$ 로 설정하여 약 1%만 이상치로 허용)
- 새로운 이미지가 입력되면 동일한 전처리와 특징 추출 과정을 거친 후, 학습된 모델을 통해 해당 이미지가 정상인지 여부를 예측하였습니다.

결과

본 프로젝트의 Step 6 공정은 불량 데이터 확보가 어렵고 패턴이 비정형적인 난제(Cold Start Problem)가 있었습니다. 이를 해결하기 위해 지도 학습 기반의 분류 (Classification) 대신, 정상 데이터 분포만을 학습하는 OneClassSVM을 도입했습니다.

특히 단순 픽셀 데이터 학습이 아닌, VGG16을 Feature Extractor로 활용해 고차원 이미지 정보를 저차원 특징 벡터로 압축·변환함으로써, 적은 데이터로도 강건한(Robust) 이상 탐지 파이프라인을 구축할 수 있었습니다.

CNN

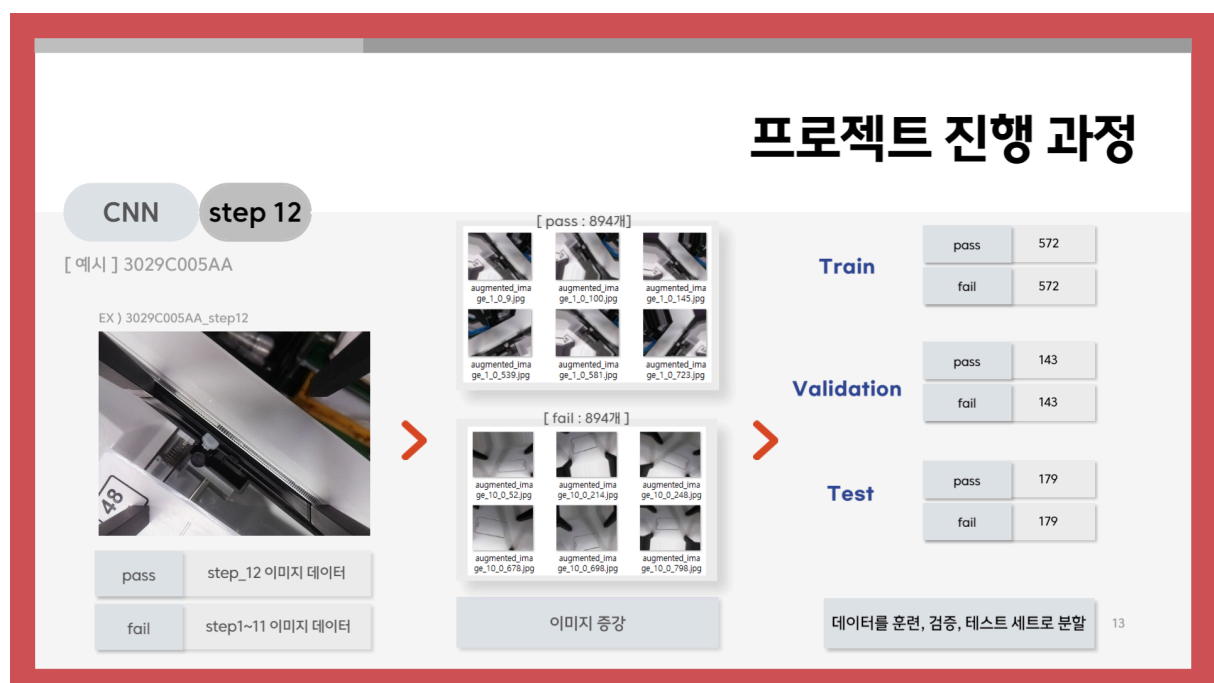
CNN 기반 검사 모델 도입

Step 12 단계에서는 기존에 사용했던 SIFT, Template Matching, One-Class SVM 기반 방식들이 아래와 같은 한계를 보였습니다.

- **SIFT**는 조명, 반사 등에 민감하여 특징점이 제대로 검출되지 않거나, 엉뚱한 위치를 매칭하는 경우가 발생했습니다.
- **Template Matching**은 정해진 형태와 위치에만 민감하게 반응하기 때문에, 미세한 위치 변화나 회전이 있는 경우 정확도가 급격히 저하되었습니다.
- **One-Class SVM**은 정상 데이터 기반 이상 탐지에는 유용했지만, 다양한 불량 유형을 모두 탐지하기엔 분포 표현력이 부족했습니다.

이러한 이유로 기존 방식만으로는 안정적인 검사 수행이 어렵다고 판단하였고,

시각적 패턴을 학습하여 다양한 변형에도 강인한 CNN 기반 이진 분류 모델을 도입하게 되었습니다.



1. 데이터 구성 및 전처리

- Step 12 영역에서 촬영된 이미지 데이터를 pass/fail로 라벨링하고, **데이터 증강을 통해 클래스 균형을 맞추었습니다.**
- 전체 데이터를 **Train/Validation/Test 세트로 분할**하여 모델의 일반화 성능을 검증할 수 있도록 구성하였습니다.

프로젝트 진행 과정

CNN

step 12

[예시] 3029C005AA

```
# 모델 구성
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(256, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(512, activation='relu'),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])
```

Optimizer	Adam
Loss Function	BinaryCrossentropy
Metrics	accuracy
epochs	5
batch_size	32

14

2. 모델 아키텍처 및 학습 설정

- CNN 모델은 4개의 Conv2D 계층과 MaxPooling, Fully Connected Layer로 구성되었으며, Optimizer는 **Adam**, Loss는 **Binary Crossentropy**, 평가 지표는 **accuracy** 를 사용하여 학습을 진행하였습니다.

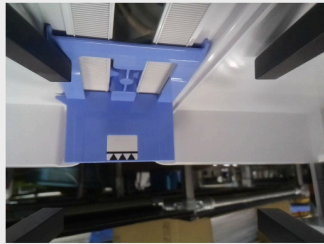
프로젝트 진행 과정

CNN

step 12

[예시] 3029C005AA / 2ET01395 / step_7.jpg

[step 7 이미지]



```
Found 358 images belonging to 2 classes.  
12/12 ————— 6s 472ms/step - accuracy: 0.9958 - loss: 0.0482  
테스트 정확도: 0.994413435459137  
1/1 ————— 0s 42ms/step  
일치율: [[0.00942084]]
```

Prediction result for the image: fail

15

모델 성능 및 결과

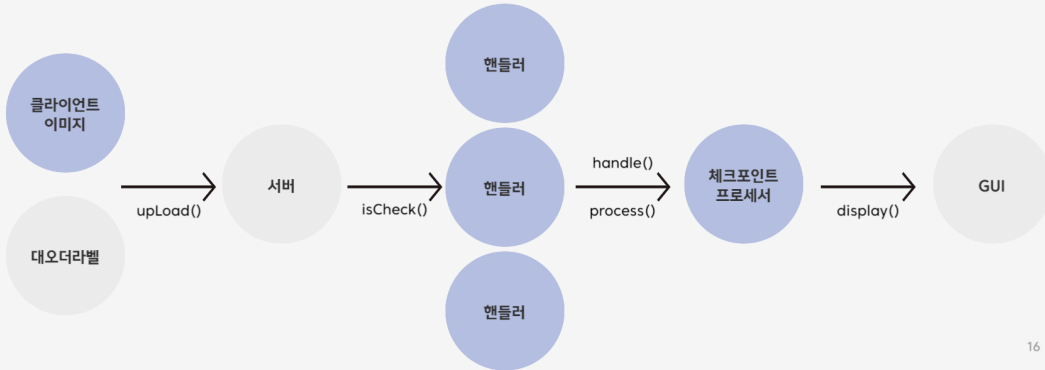
- 테스트 정확도 **99.4%**를 기록하며, 기존 방식 대비 훨씬 높은 신뢰도를 확보하였습니다.
- 실제 검사 이미지에 대해 **fail 여부를 높은 정확도로 예측**하며, 전처리 없이도 다양한 시각적 변형에 유연하게 대응할 수 있었습니다.
- CNN을 통한 분류 결과는 **검사 일관성 향상과 오탐/미탐 감소**에 직접적으로 기여하였으며,
이후 다른 검사 단계에도 동일한 구조의 모델을 확장 적용할 수 있는 가능성을 확인하였습니다.

확장성 있는 검사 프로세스 설계

프로젝트 진행 과정

새로운 기종 추가를 고려한 시스템 구조 설계

- 시스템의 전체적 개요



새로운 기종 추가를 고려한 시스템 구조 설계

다양한 기종과 검사 항목의 확장 가능성을 고려하여, **모듈화된 이미지 검사 처리 파이프라인**을 설계하였습니다.

- 클라이언트에서 업로드한 이미지와 대오더 라벨 정보를 **서버에서 수신**
- 서버는 기종에 따라 적절한 **핸들러(handler)**를 선택하고, 각 핸들러는 해당 검사 프로세스를 **체크포인트 프로세서**에 전달합니다.
- 체크포인트 프로세서는 단계별 이미지 분석 결과를 **GUI에 시각화**하여 사용자에게 전달합니다.

핸들러 단위의 구조를 통해 **기종별 커스터마이징이 유연하게** 가능하며,

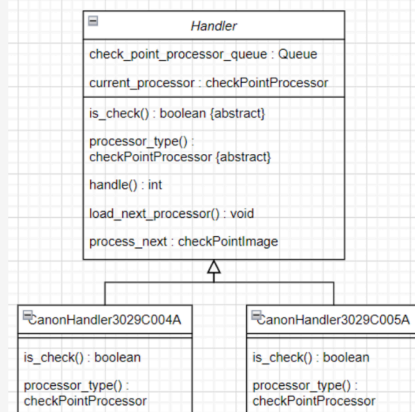
새로운 검사 항목 추가 시에도 **핵심 로직 변경 없이 모듈만 추가**하면 되도록 설계하였습니다.

프로젝트 진행 과정

새로운 기종 추가를 고려한 시스템 구조 설계

- 핸들러 클래스 구조도

추상 클래스와 추상 메서드를 사용하여
새로운 기종에 대한 핸들러를 쉽게 추가할 수 있도록 설계



17

핸들러 클래스 구조 설계

새로운 기종이 추가될 때마다 시스템 로직을 수정하지 않고,

핸들러만 새롭게 정의하여 확장할 수 있도록 추상 클래스 기반 구조로 설계하였습니다.

- **Handler** 클래스를 상위 클래스로 정의하고,
is_check(), **processor_type()** 등의 추상 메서드(**abstract method**)를 선언하였습니다.
- 각 기종에 대응되는 핸들러 클래스(CanonHandler3029C004A 등)는 이를 상속하여 오버라이딩함으로써, 기종별 고유한 처리 로직을 쉽게 추가할 수 있도록 구성하였습니다.
- 검사 순서는 **Queue** 형태의 프로세서 리스트로 관리되어, 기종마다 유연한 검사 흐름 제어가 가능합니다.

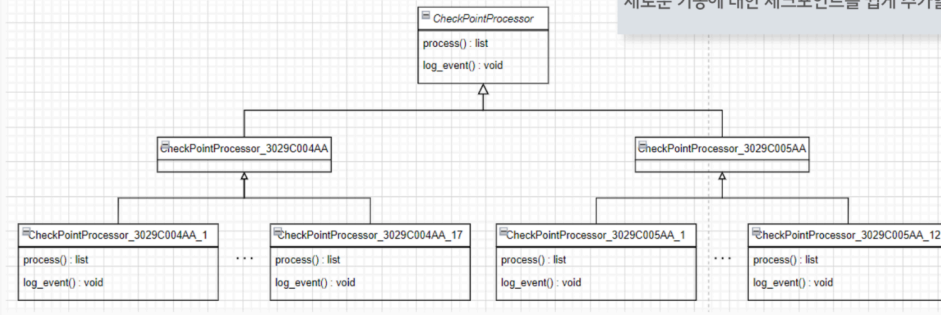
이 구조를 통해 유지보수성과 확장성을 모두 확보하였으며,

추후 신규 기종이 도입될 경우에도 핸들러 클래스만 추가하면 시스템에 즉시 반영할 수 있도록 설계하였습니다.

프로젝트 진행 과정

새로운 기종 추가를 고려한 시스템 구조 설계

- 체크포인트 프로세서 클래스 구조도



추상 클래스와 추상 메서드를 사용하여
새로운 기종에 대한 체크포인트를 쉽게 추가할 수 있도록 설계

18

체크포인트 프로세서 클래스 구조 설계

다양한 기종과 검사 단계에 대응하기 위해,

공통된 인터페이스를 기반으로 한 추상 클래스 구조를 설계하였습니다.

- `CheckPointProcessor` 추상 클래스에 `process()` 와 `log_event()` 메서드를 정의하여,
모든 검사 단계에서 동일한 호출 흐름을 유지할 수 있도록 구성하였습니다.
- 각 기종 및 검사 단계별로 세분화된 `CheckPointProcessor_기종명_번호` 클래스를 상속 구현하여,
개별 검사 로직을 분리하고 재사용이 가능한 구조로 확장하였습니다.
- 신규 기종이 도입될 경우, 해당 기종에 맞는 프로세서 클래스만 추가하면 전체 시스템 구조에 영향을 주지 않고 손쉽게 검사 흐름을 적용할 수 있습니다.

구현 영상

[attachment:5130f21a-4f3d-4e52-9f63-a773f1f39929:캐논_최종.mp4](#)

최종 PPT

[CanonProject.zip](#)